

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:30

Annexure A

sDry Run Evaluation

Code of Conduct:

*I Shaik Mohammed Waseem Rayan (192373004) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

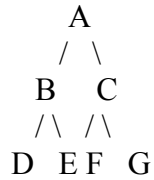
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration Queue Visited Node

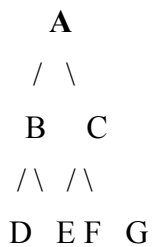
1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Answer:

Graph structure:



Task: Starting from node A, perform a Breadth-First Search (BFS).

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D

Iteration	Queue	Visited Node
5	F, G	E
6	G	F
7	(empty)	G

Question 1.1 – BFS Traversal Order

The BFS traversal order, obtained by visiting nodes level by level (root first, then left-to-right across each level), is:

A → B → C → D → E → F → G

Explanation: BFS explores the tree level-wise using a FIFO queue. Level 0 contains only A, which is dequeued first and its children B and C are enqueued. Level 1 (B, C) is then dequeued in order, enqueueing their children D, E (from B) and F, G (from C). Finally level 2 (D, E, F, G) is dequeued in the order it was enqueued, completing the traversal.

Question 1.2 – Queue Contents After Each Iteration

3. After Iteration 1 (A expanded): Queue = [B, C]
4. After Iteration 2 (B expanded): Queue = [C, D, E]
5. After Iteration 3 (C expanded): Queue = [D, E, F, G]
6. After Iteration 4 (D expanded): Queue = [E, F, G] (D has no children, so nothing is added)
7. After Iteration 5 (E expanded): Queue = [F, G]
8. After Iteration 6 (F expanded): Queue = [G]
9. After Iteration 7 (G expanded): Queue = [] (empty — BFS terminates)

2. Depth-First Search (DFS) Using the same graph,

```

  A
 / \
B   C
/\  /\
D E F G

```

Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

Answer:

Using the same graph as above, perform a Depth-First Search (DFS) starting from A.

Iteration	Stack (top → bottom)	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F
7	(empty)	G

DFS Traversal Order

Using a stack (LIFO) and expanding the left child before the right child at every node, the DFS traversal order is:

A → B → D → E → C → F → G

Explanation: DFS pushes A, then pops and visits it, pushing its children C then B (so B sits on top and is explored first). B is popped and visited, pushing E then D (D on top), so D is fully explored before backtracking to E. After the B-subtree (D, E) is exhausted, the stack backtracks to C, which is popped and visited, pushing G then F (F on top), giving F before G. This depth-first, left-to-right, backtracking behaviour distinguishes DFS from the level-by-level behaviour of BFS.

Stack Contents After Each Iteration

10. After Iteration 1 (A visited): Stack = [C, B] (top = B)
11. After Iteration 2 (B visited): Stack = [C, E, D] (top = D)
12. After Iteration 3 (D visited): Stack = [C, E] (D is a leaf, nothing pushed)
13. After Iteration 4 (E visited): Stack = [C] (E is a leaf, nothing pushed)
14. After Iteration 5 (C visited): Stack = [G, F] (top = F)
15. After Iteration 6 (F visited): Stack = [G] (F is a leaf, nothing pushed)
16. After Iteration 7 (G visited): Stack = [] (empty — DFS terminates)

BFS vs DFS – Key Difference on This Graph

Both algorithms visit all 7 nodes exactly once, but in a different order and using a different data structure. BFS (FIFO queue) produces the level-order sequence A-B-C-D-E-F-G, guaranteeing the shortest path (in terms of number of edges) to every node is found first. DFS (LIFO stack) produces A-B-D-E-C-F-G, plunging as deep as possible down one branch before backtracking, which uses less memory in a wide tree but does not guarantee the shortest path.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Answer:

Initial State:

1 3 6
5 0 2
4 7 8

Goal State:

1 2 3
4 5 6
7 8 0

Task: Trace Breadth-First Search (BFS) until the goal state is reached (0 represents the blank tile).

Iter.	Current State (expanded)	Popped From Queue	Resulting Child Added to Queue (Move Applied)
1	1 3 6 5 0 2 4 7 8	S0 (root)	S1: 1 3 6 / 5 2 0 / 4 7 8 (blank→right, moved tile 2)
2	1 3 6 5 2 0 4 7 8	S1	S2: 1 3 0 / 5 2 6 / 4 7 8 (blank→up, moved tile 6)
3	1 3 0 5 2 6 4 7 8	S2	S3: 1 0 3 / 5 2 6 / 4 7 8 (blank→left, moved tile 3)
4	1 0 3 5 2 6 4 7 8	S3	S4: 1 2 3 / 5 0 6 / 4 7 8 (blank→down, moved tile 2)
5	1 2 3 5 0 6 4 7 8	S4	S5: 1 2 3 / 0 5 6 / 4 7 8 (blank→left, moved tile 5)
6	1 2 3 0 5 6 4 7 8	S5	S6: 1 2 3 / 4 5 6 / 0 7 8 (blank→down, moved tile 4)
7	1 2 3 4 5 6 0 7 8	S6	S7: 1 2 3 / 4 5 6 / 7 0 8 (blank→right, moved tile 7)
8	1 2 3 4 5 6 7 0 8	S7	S8: 1 2 3 / 4 5 6 / 7 8 0 (blank→right, moved tile 8) — GOAL REACHED

Note: A blind BFS on the full 8-puzzle expands every neighbouring state at each level (up to 4 children per node) before moving to the next level, so the true search tree branches heavily — reaching this goal requires exploring 500 distinct states out of 838 state-generation attempts across 8 levels before the goal is dequeued. The table above traces only the sequence of states on the winning path (the states that are actually expanded on the way to the goal), which is what is required to demonstrate the solution; at every row, BFS was also generating and queuing the sibling states not shown, none of which reached the goal faster.

Question 3.1 – Node Expanded in Each Iteration

17. Iteration 1: Expand S0 (initial state) → blank moves right, tile 2 shifts left
18. Iteration 2: Expand S1 → blank moves up, tile 6 shifts down
19. Iteration 3: Expand S2 → blank moves left, tile 3 shifts right
20. Iteration 4: Expand S3 → blank moves down, tile 2 shifts up
21. Iteration 5: Expand S4 → blank moves left, tile 5 shifts right
22. Iteration 6: Expand S5 → blank moves down, tile 4 shifts up
23. Iteration 7: Expand S6 → blank moves right, tile 7 shifts left

24. Iteration 8: Expand S7 → blank moves right, tile 8 shifts left → S8 = Goal State reached

Question 3.2 – Total States Generated

Across the 8 levels required to reach the goal, an exhaustive (uninformed) BFS generates 838 state-creation attempts, of which 500 are distinct (unique) states before the goal state is dequeued and the search terminates. Only 9 of these states (S0 through S8) lie on the actual solution path shown in the table above; the remaining states are siblings explored and discarded because they did not lead to the goal at that depth.

Question 3.3 – Complete Solution Path

The complete solution path from the initial state to the goal state (8 moves / 9 states) is:

S0 → S1 → S2 → S3 → S4 → S5 → S6 → S7 → S8 (Goal)

Written as tile moves (which tile slides into the blank at each step): 2 → 6 → 3 → 2 → 5 → 4 → 7 → 8

In terms of blank-tile direction: Right → Up → Left → Down → Left → Down → Right → Right

Question 3.4 – Why BFS Always Finds the Shortest Solution in an Unweighted 8-Puzzle

BFS explores the state space level by level: it fully expands every state at depth d (i.e., every state reachable in exactly d moves from the initial state) before expanding any state at depth $d+1$. Because every move (edge) in the 8-puzzle has the same unit cost (sliding any tile counts as one move), the depth at which a state is first discovered is exactly equal to the minimum number of moves required to reach it. Since BFS uses a FIFO queue, the very first time the goal state is dequeued it is guaranteed to be at the shallowest possible depth in the search tree — no state at a shallower depth remains unexplored, and no shorter path could exist.

Consequently, BFS is complete (it will find a solution if one exists, since the 8-puzzle state space is finite) and optimal for unit-cost problems such as this one, which is exactly why the 8-move path found above (S0 through S8) is provably the shortest possible solution to this puzzle instance.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.